Sources of Uncertainty in AI Evaluation

Why your scorecard lies — and what to do about it

Designing Large Scale AI Systems · Spring 2026

The Setup

AI System → LLM Judge → 1-5 Scores
4-Metric Scorecard: Correctness, Helpfulness, Tone, Completeness

Nine Sources + One Layer

1. Sampling Noise
2. Sampling Bias
3. Overfitting
4. Multiple Hypothesis
5. Variance Across Domains
6. Choice of Metric
7. Temporal Drift
8. Prompt Brittleness
9. LLM Evolution

The Judge Layer: Cross-cutting uncertainty layer

#1

Sampling Noise

Your scorecard wobbles

The Problem: Small n, Big Wobbles

Repeated evaluations of the same system give wildly different scores.

Run 1: Correctness 4.2, Helpfulness 3.9, Tone 4.5, Completeness 3.4
Run 2: Correctness 3.8, Helpfulness 4.3, Tone 3.9, Completeness 4.1
Run 3: Correctness 4.4, Helpfulness 4.0, Tone 4.2, Completeness 3.7

▶ Interactive: sampling_noise.html

The Fix: Confidence Intervals

Beta Posteriors (binarize ≥4):
Small n: wide interval | Large n: narrow spike

Bootstrap on raw 1-5 scores:
Resample with replacement, compute distribution of mean

Key Insight: Report confidence intervals, never bare point estimates.
Variance shrinks as 1/n — more data is a clean fix here.

Does More Data Help?

n = 30
CI: ±0.4

n = 100
CI: ±0.2

n = 500
CI: ±0.08

YES ✓ Uncertainty collapses as √(1/n)

#2

Sampling Bias

Your test set doesn't look like production

The Problem: Wrong Population Mix

Test Set: 95% English, 5% Italian
Production: 60% English, 40% Italian

Test Scorecard: Correctness 4.2
Production Reality: Correctness 3.2

Language is one axis. Bias lurks on any dimension:
Synthetic vs real data, simple vs complex queries,
formal vs casual tone, domain-specific topics

The Fix: Know Your Production

1. Know your production distribution. Measure it. Don't assume.

2. Stratified sampling: sample proportionally to production mix.

3. Per-segment reporting: never trust only the aggregate.

4. Validate on real production data when possible.

Does More Data Help?

NO ✗ More biased data is still biased.

10,000 wrong samples < 100 right samples

More data helps ONLY if it's representative.
The fix is in the sampling design, not the sample size.

#3

Overfitting

You looked at the test data

The Problem: Information Leakage

You iterated on the prompt using dev examples.
You read the examples, understood patterns,
and encoded them into the prompt.

Even copy-pasted dev examples as few-shot examples.

Dev score climbs steadily.
Held-out test score: barely moves, then stalls.

Dev: not trustworthy after many iterations.

The Lab: Live Iteration

Task B1 — Judge calibration (Prometheus/FeedbackBench)
Tune an LLM judge prompt. Dev set: 50 examples.
Test set: 200 examples. Watch dev improve, test plateau.

Task B2 — Structured extraction (TWEETSUMM)
Extract fields from support tickets. Tune prompt iteratively.
Rules that work on dev don't generalize to test.

The Gap: When did you stop trusting the dev number?

Does More Data Help?

PARTIALLY Larger dev set is harder to overfit.
But with enough iterations, you'll overfit any finite set.

Real fix: Process discipline.
Held-out test set you only touch once.

More data buys you time, not safety.

#4

Multiple Hypothesis Testing

You picked the luckiest variant

The Problem: Selection Bias

You designed 10 prompt variants independently.
You were disciplined — never looked at test data.
Then you evaluated all 10 on the same test set.
Variant #7 scored highest. You ship it.

But: the test set had sampling noise.
Variant #7 was just the luckiest.
On fresh data, it often performs worse
than variants you discarded.

Key Insights: Selection Bias Scales with N

More variants = bigger bias.

N=2 variants: ±0.05 inflation
N=10 variants: ±0.15 inflation
N=50 variants: ±0.35 inflation

The more thorough your search,
the more you fool yourself.

The Fix: Two-Stage Evaluation

Stage 1: Use dev set to select winner

Stage 2: Use fresh test set to measure winner's true performance

Alternatives:
- Bonferroni / Holm-Bonferroni correction
- Pre-register which variant you're testing
- Report adjusted CI for best-of-N

Does More Data Help?

PARTIALLY More data shrinks per-variant noise.
Selection bias gets smaller, never reaches zero.

Worse: tighter CIs make noise look 'significant',
so you're more confident in a biased pick.

Real fix: process (two-stage, corrections),
not just more data.

#5

Variance Across Domains

The aggregate lies

The Problem: Simpson's Paradox

Overall Correctness: 4.0 (looks good)

Per-segment reality:
  Finance queries: 4.6 → happy
  Medical queries: 3.3 → churns
  Legal queries: 3.8 → confused

Simpson's Paradox: System A beats B overall,
but B beats A in *every single segment*.

The Fix: Per-Segment Everything

1. Always report per-segment scores.
Never trust only the aggregate.

2. Define minimum acceptable performance per segment.
A system at 4.5 finance / 2.0 medical is broken.

3. Set SLAs per segment, per customer.

4. Improve the system itself to reduce real variance.
Make it more adaptive to different domains.

Does More Data Help?

NO ✗ More data gives tighter estimates.
Doesn't reduce the variance between segments.

Variance is a property of the world:
Your system handles finance well, medical poorly.

Only system improvements reduce the variance.
Measurement precision ≠ system improvement.

#6

Choice of Metric

The winner depends on how you ask

The Problem: Same Data, Different Winners

System A vs B — 200 queries, same judge

Mean: A 4.1 vs B 3.9 → A wins
Median: A 4.0 vs B 4.0 → Tie
% scoring ≥4: A 62% vs B 68% → B wins
5th percentile: A 1.8 vs B 2.5 → B wins

A has higher average. B is more consistent.
Which is 'better'? The data didn't change. The question changed.

The Fix: Pre-Commit

Pre-register before evaluation:
- Primary metric (mean? threshold? percentile?)
- Specific aggregation method
- Success criteria

Metric hierarchy: primary metric + guardrails.

Document why you chose the metric.
What user outcome does it proxy for?

The metric choice is part of system design,
not an afterthought.

#7

Temporal Drift

Your number has an expiration date

The Problem: The World Changes

Customer support bot — monthly evaluations:

January: Correctness 4.2 → ship it
February: Correctness 4.1
March: Correctness 3.8 → hmm
April: Correctness 3.5 → panic

Code never changed. World changed.
New product launch → queries the system was never designed for.
Your January evaluation tells you about January. Nothing about April.

The Fix: Continuous Monitoring

1. Build monitoring into the system from day one.

2. Automated periodic evaluation on fresh production data.

3. Drift detection: alert when scores drop or shift too much.

4. Version everything (data, prompts, models)
   to diagnose what drifted.

Does more data help? NO — more old January data
tells you about January. Need fresh data over time.

#8

Prompt Brittleness

One comma changed your accuracy by 20%

Real Research Findings

Reorder few-shot examples: 54% → 93% accuracy
Change single delimiter char: ±23% accuracy
Punctuation/whitespace only: 9.4% → 54.9% (6x swing!)
Move answer option positions: Model ranking changes
Formatting in few-shot: Up to 76 accuracy points

Production incident: Changing "strictly valid JSON" to "clean, parseable JSON" silently broke parsers.

These aren't corner cases. They're typical.

The Fix: Sensitivity Testing

Before shipping, try N rephrasings.
Measure variance in scores.

Report score as a range, not a single number.
Best / worst across rephrasings.

Robustness as a selection criterion:
Prefer prompts where small changes → small score changes.

Pin prompt versions exactly.
Treat any edit as a new deployment.

Does More Data Help?

NO ✗ More data makes each prompt variant more precise, but doesn't reduce brittleness.

The sensitivity between prompts is a property of the model, not the measurement.

You can measure it more precisely with more data. The gap between variants stays.

#9

LLM Evolution

The ground shifted under you

The Problem: You Don't Own Your Model

The provider updated the weights silently.
Your prompt, test set, code: unchanged.
Scores shifted.

Key example — Chen et al. (2023):
GPT-4 on 'identify prime numbers'
March 2023: 97.6% accuracy
June 2023: 2.4% accuracy

Catastrophic regression. Same task, same prompt.
22.9% of model updates cause prompt regressions (>20% drops).

The Fix: Version Discipline

Pin model versions (gpt-4-0613, not gpt-4).

Automated regression tests on a schedule.
Catch degradation early.

Fallback strategy: if new version regresses, roll back.

Treat every model version change as a deployment.
Full re-evaluation required.

Monitor provider changelogs and deprecation notices.

The Judge Layer

Every source applies again

Every Measurement Goes Through the Judge

The judge (LLM or human) is itself an imperfect system.
Every uncertainty source applies AGAIN at the judge level.

Judge issues:
- Noise: same judge, same output, different score
- Bias: systematically generous or harsh
- Subjectivity: experts disagree on 'what is a 4?'
- Brittleness: judge prompt rewording → different scores
- Evolution: judge model gets updated

Judge uncertainties MULTIPLY with others. They don't add.

The Compounding Effect

Perfect judge scenario:
Sampling noise alone: CI ±0.2

Add judge noise (65% self-agreement):
Each data point is itself uncertain: CI ±0.4

Add judge bias (systematically +0.3):
CI still ±0.4, but center shifts to 4.4

Add judge subjectivity:
There isn't even a single 'true' value to estimate.

Noisy judge doesn't just add fuzz — it doubles/triples uncertainty.

Judge Issue: Subjectivity

Some things are genuinely subjective.
"Is this response helpful?" Reasonable experts disagree.
There isn't a ground truth to converge to.

Example: 20 outputs, 5 human experts score Helpfulness
Some outputs: all experts agree (4-5)
Others: range from 2 to 5

If experts disagree, what is the 'right' score?
Your ground truth is a social construction, not a fact.
The LLM judge adopted one subjective standard.

Uncertainty Compounds

It does not cancel out

Watch the Uncertainty Grow

What's included	Score	Uncertainty
Perfect judge, infinite data	4.10	Exact
+ Sampling noise (n=100)	4.10 ± 0.15	CI narrows
+ Judge noise (65% agreement)	4.10 ± 0.35	Each point uncertain
+ Judge bias (systematically +0.3)	4.40 ± 0.35	Shifted center
+ Sampling bias (wrong population)	4.40 ± 0.35	Truth outside CI
+ Prompt brittleness (±0.3)	3.4–4.8	Multiple values
+ Model evolution (judge updated)	???	Instrument changed

Anti-Intuitions Students Need to Develop

WRONG: Errors cancel out
RIGHT: Variances add, biases compound

WRONG: More data fixes everything
RIGHT: Only fixes #1. Doesn't fix bias, judges, brittleness, drift

WRONG: If the number looks precise, it is precise
RIGHT: Precision ≠ accuracy. A very precise wrong number is worse

WRONG: We measured 4.1, so it's approximately 4.1
RIGHT: 4.1 = output of this judge, prompt, sample, model, moment

The Key Takeaway

The scorecard is not a measurement of your system.

It's a measurement of your system
as seen by THIS judge,
with THIS prompt,
on THIS data,
at THIS moment in time.

Every one of those qualifiers hides uncertainty.
And the uncertainties do not cancel —
they compound.

Does More Data Help? Summary

YES (#1): Sampling noise — variance shrinks as 1/n

PARTIAL (#3, #4): Overfitting & MHT — harder but not impossible

NO (#2, #5, #6, #7, #8, #9): Bias, drift, brittleness, evolution
More data from the same biased source is still biased.
More old data tells you about the past.

More data reduces sampling noise.
Everything else requires better judges, better design,
better discipline, or accepting uncertainty as real.

Interactive Tools & Playgrounds

Explore the sources interactively:

▶ sampling_noise.html — adjust n, watch CI collapse
▶ sampling_bias.html — mix biased samples, see the gap
▶ multiple_hypothesis_testing.html — N variants, N winners
▶ variance_across_domains.html — Simpson's paradox live
▶ choice_of_metric.html — same data, different winners
▶ temporal_drift.html — monthly scores over time
▶ stacking.html — watch uncertainty compound

Go play with it. Get a feel for how these sources hide in your measurements.

"The goal of science is not to produce graphs,
but to produce understanding.
We measure to learn, not to celebrate
the precision of the measurement itself."

— In spirit of Tukey, Box, Ioannidis,
and everyone who's shipped a model
and watched it fail quietly in production